CAHIER DES CHARGES

Coffre-fort Numérique Sécurisé

2026

Table des matières

1. Contexte du projet....2
2. Vision et objectifs....3
2.1 Objectif principal....3
2.2 Objectifs pédagogiques....3
2.3 Livrable attendu....3
3. Organisation du projet....4
4. Périmètre fonctionnel....4
4.1 Gestion des comptes et de l'authentification....4
4.2 Stockage sécurisé des fichiers....4
4.3 Partage et diffusion....5
4.4 Journalisation....5
4.5 Gestion des quotas....5
5. Interfaces utilisateurs....5
5.1 Client lourd JavaFX....6
5.2 Client Web public....6
6. Exigences non fonctionnelles....7
6.1 Sécurité réseau....7
6.2 Journalisation....7
6.3 Sauvegardes....7
6.4 Performance....7
6.5 Qualité....7
6.6 RGPD....7
7. Architecture technique....8
7.1 Stack technologique....8
7.2 Modèle de données proposé....8
8. Contrat d'API....10
8.1 Authentification....10
8.2 Dossiers et fichiers....10
8.3 Partages....10
8.4 Quotas et activité....10
8.5 Convention d'erreurs....10
9. Sécurité détaillée....11
9.1 Chiffrement au repos....11
9.2 Authentification et session....11
9.3 Liens de partage sécurisés....11
9.4 Sécurité HTTP....11
10. Tests....12
11. Planning sur 7 jours....12
12. Critères d'acceptation du MVP....13
13. Livrables attendus....13
14. Perspectives d'évolution....14
15. Conclusion....14

1. Contexte du projet

Le projet ***CryptoVault*** s'inscrit dans le cadre de la formation BTS SIO, dans une logique de mise en situation professionnelle.
Il est demandé par l'établissement **IRIS MediaSchool Nice**, afin de permettre aux étudiants de concevoir une application complète mobilisant des compétences en développement logiciel, en sécurité informatique, en gestion de projet et en travail collaboratif.

Le projet consiste à concevoir un ***coffre-fort numérique sécurisé***, accessible via une API REST, un client lourd JavaFX et une interface Web dédiée aux liens de partage publics.

Il s'appuie sur la base technique fournie dans le TP pédagogique ***Mini coffre-fort REST (Slim + Medoo),*** avec pour objectif de faire évoluer un simple système CRUD de fichiers vers une solution complète, sécurisée, structurée et maintenable.

Le projet est mené par une équipe de 2 étudiants, avec un livrable final attendu sous la forme d'un MVP fonctionnel, accompagné de sa documentation, de ses tests et d'une démonstration.

2. Vision et objectifs

2.1 Objectif principal

Développer une application permettant à un utilisateur de :

- stocker des fichiers de manière sécurisée,
- organiser ses données dans une arborescence de dossiers,
- partager des fichiers ou des dossiers via des liens sécurisés,
- suivre l'activité liée à ses fichiers et à ses partages,
- gérer l'espace de stockage attribué à son compte.

2.2 Objectifs pédagogiques

Le projet doit permettre de :

- concevoir et développer une ***API REST sécurisée*****,**
- mettre en place une architecture complète intégrant back-end, client lourd et interface Web,
- implémenter des mécanismes de sécurité modernes (***JWT, chiffrement, contrôle des accès***),
- travailler en binôme, avec coordination des développements,
- produire une documentation technique et utilisateur,
- mettre en œuvre des tests unitaires, d'intégration et de validation,
- structurer un projet de manière professionnelle, de l'analyse du besoin à la démonstration finale.

2.3 Livrable attendu

Le livrable attendu comprend :

- un MVP fonctionnel couvrant le périmètre défini,
- une documentation technique et utilisateur complète,

- une suite de tests adaptée aux fonctionnalités principales,
- une démonstration fil rouge de bout en bout :
    - création de compte,
    - authentification,
    - création d'un dossier,
    - upload de fichier,
    - partage,
    - téléchargement,
    - création d'une nouvelle version,
    - visualisation des quotas.

3. Organisation du projet

Le projet est réalisé en **binôme**, avec une répartition des tâches permettant de couvrir l'ensemble du développement.

Les responsabilités portent notamment sur :

- le développement back-end (API REST, sécurité, base de données),
- le développement du client JavaFX,
- le développement de l'interface Web publique,
- la mise en place des tests, de la documentation et du suivi du projet.

Le travail s'effectue de manière collaborative à l'aide de :

- GitHub pour la gestion de versions,
- échanges réguliers entre les membres du binôme,
- validation progressive des fonctionnalités,
- organisation des tâches selon un planning défini.

4. Périmètre fonctionnel

4.1 Gestion des comptes et de l'authentification

L'application doit permettre :

- la création de compte,
- l'authentification par **JWT** (sessions stateless),
- la gestion de deux rôles :
    - **Utilisateur** : propriétaire d'un coffre-fort numérique,
    - **Administrateur** : gestion des quotas et supervision,
- l'application d'une politique de mot de passe sécurisée,
- le hachage des mots de passe avec Bcrypt.

4.2 Stockage sécurisé des fichiers

L'application doit permettre :

- l'upload de fichiers,
- le téléchargement de fichiers,
- la suppression de fichiers,
- l'organisation des données dans des dossiers hiérarchiques,
- le versionnage des fichiers.

Le système doit assurer :

- le chiffrement au repos des fichiers,
- l'utilisation de AES-256-GCM pour le contenu des fichiers,
- la protection des clés de chiffrement au moyen d'une enveloppe sécurisée,
- la vérification de l'intégrité à l'aide d'un checksum.

Chaque remplacement d'un fichier doit créer une nouvelle version, sans écrasement de la version précédente.

4.3 Partage et diffusion

L'application doit permettre :

- la génération de liens de partage sécurisés,
- le paramétrage de ces liens :
    - date d'expiration,
    - nombre maximal de téléchargements,
- la révocation immédiate d'un lien,
- l'accès à une page publique de téléchargement.

Les liens doivent pouvoir viser :

- un fichier,
- un dossier, selon les règles retenues par l'équipe.

4.4 Journalisation

Le système doit enregistrer les événements liés aux partages et téléchargements, notamment :

- la date et l'heure,
- l'adresse IP,
- le user-agent,
- le succès ou l'échec de l'opération.

4.5 Gestion des quotas

L'application doit intégrer :

- une limite d'espace par utilisateur,
- un quota par défaut configurable,
- des alertes visuelles à 80 % et 100 %,

- le blocage des uploads en cas de dépassement de quota.

5. Interfaces utilisateurs

5.1 Client lourd JavaFX

Le client JavaFX constitue l'interface principale de gestion du coffre-fort. Il doit permettre :

- l'authentification,
- la navigation dans l'arborescence des dossiers,
- la consultation des fichiers,
- l'upload avec barre de progression,
- le téléchargement,
- le renommage, le déplacement et la suppression,
- la création de liens de partage,
- la consultation et la révocation des partages existants,
- l'affichage des quotas,
- la gestion des versions de fichiers.

Écrans attendus

Le client JavaFX doit comporter au minimum :

1. un écran de connexion,
2. un tableau de bord avec affichage du quota,
3. un explorateur de stockage,
4. une vue détail d'un fichier avec versions,
5. une fenêtre d'upload / remplacement,
6. une vue de gestion des partages,
7. une page de paramètres.

Contraintes UX

L'interface doit prévoir :

- des confirmations pour les actions destructrices,
- des indicateurs de progression pour les opérations longues,
- une gestion claire des erreurs HTTP,
- une accessibilité minimale (navigation clavier, libellés explicites, contrastes suffisants).

5.2 Client Web public

Le client Web a pour objectif de permettre l'accès à une ressource partagée via un lien sécurisé, sans authentification.

Il doit permettre :

- l'affichage des métadonnées d'un fichier ou d'un dossier partagé,
- le téléchargement de la ressource si le lien est valide,
- l'affichage d'un message précis en cas de :
    - lien expiré,

- lien révoqué,
- nombre d'usages épuisé,
- token inconnu ou malformé.

Contraintes
La page doit être :
- responsive,
- simple,
- accessible,
- non indexable par les moteurs de recherche.

6. Exigences non fonctionnelles

6.1 Sécurité réseau

- Communication en HTTP en développement local.
- Communication en HTTPS en production sur le serveur de l'école :
  https://cryptovault.iris.a3n.fr:4433
- Configuration restrictive du CORS (*Cross-Origin Resource Sharing*), limitant les requêtes aux origines, méthodes et en-têtes explicitement autorisés.
- Activation des headers de sécurité en production.

6.2 Journalisation

- Journaux structurés.
- Traçabilité des actions sensibles.
- Aucun secret (mot de passe, token, chemin privé) ne doit apparaître dans les logs.

6.3 Sauvegardes

- Procédure documentée de sauvegarde de la base de données et des fichiers chiffrés.
- Test de restauration prévu et vérifiable.

6.4 Performance

- Upload multipart.
- Téléchargement en streaming.
- Aucun fichier complet ne doit être chargé inutilement en mémoire lors du chiffrement ou du déchiffrement.
- Gestion des fichiers volumineux.

6.5 Qualité

- Tests unitaires,
- tests d'intégration,
- tests E2E/Postman/Newman,

- intégration continue avec GitHub Actions,
- validation automatique avant fusion.

6.6 RGPD

- Présence de mentions légales,
- suppression de compte prévue ou documentée,
- réflexion sur l'export des données personnelles.

7. Architecture technique

7.1 Stack technologique

Back-end

- PHP 8.2+
- Slim 4
- Medoo
- MySQL
- Firebase JWT
- OpenSSL / libsodium

Client JavaFX

- Java 17+
- JavaFX 17+
- OkHttp ou HttpClient Java
- Jackson ou Gson
- Maven ou Gradle

Client Web

- HTML
- CSS
- JavaScript
- Bootstrap 5
- Slim MVC pour les vues si nécessaire

Outils et déploiement

- Docker / Docker Compose
- GitHub
- GitHub Actions
- Postman / Newman
- variables d'environnement (.env)

7.2 Modèle de données proposé

Table users

- id
- email
- pass_hash
- quota_total
- quota_used
- is_admin
- created_at

Table folders

- id
- user_id
- parent_id
- name
- created_at

Table files

- id
- user_id
- folder_id
- original_name
- mime
- size
- created_at

Table file_versions

- id
- file_id
- version
- stored_name
- iv
- auth_tag
- key_envelope
- checksum
- created_at

Table shares

- id
- user_id
- kind
- target_id
- label
- expires_at
- max_uses

- remaining_uses
- is_revoked
- created_at

Table downloads_log

- id
- share_id
- version_id
- downloaded_at
- ip
- user_agent
- success

8. Contrat d'API

Le fichier ***openapi.yaml*** constitue la source de vérité des échanges entre les différentes parties du projet.

8.1 Authentification

- POST /auth/register
- POST /auth/login

8.2 Dossiers et fichiers

- GET /folders
- POST /folders
- DELETE /folders/{id}
- GET /files?folder={id}
- POST /files
- POST /files/{id}/versions
- GET /files/{id}
- DELETE /files/{id}
- GET /files/{id}/download

8.3 Partages

- POST /shares
- GET /shares
- POST /shares/{id}/revoke
- GET /s/{token}
- POST /s/{token}/download

8.4 Quotas et activité

- GET /me/quota
- GET /me/activity (optionnel)

8.5 Convention d'erreurs

Les erreurs doivent suivre le format JSON suivant :

{ "error": "message", "code": "CODE_APPLICATIF" }

Codes principaux attendus :

- 200 / 201 / 204 : succès
- 400 : requête invalide
- 401 : non authentifié
- 403 : accès interdit
- 404 : ressource introuvable
- 409 : conflit
- 410 : lien révoqué ou expiré
- 413 / 422 : quota ou validation
- 429 : trop de requêtes
- 500 : erreur serveur

9. Sécurité détaillée

9.1 Chiffrement au repos

- chiffrement AES-256-GCM,
- clé de contenu aléatoire par version,
- IV aléatoire,
- auth_tag stocké,
- checksum SHA-256,
- nom physique de fichier généré de manière unique.

9.2 Authentification et session

- mots de passe hachés avec Bcrypt,
- JWT avec expiration,
- refresh token optionnel,
- token JavaFX stocké uniquement en mémoire,
- déconnexion automatique en cas d'expiration.

9.3 Liens de partage sécurisés

- token signé,
- vérification de validité,
- prise en compte de l'expiration,

- prise en compte du nombre d’usages,
- décrément atomique du compteur d’usages,
- retour 410 pour les liens expirés ou révoqués.

9.4 Sécurité HTTP

- HTTPS en production,
- headers de sécurité,
- CORS maîtrisé,
- rate limiting,
- page publique non indexable.

10. Tests

Le projet doit intégrer :

- des tests unitaires des contrôleurs,
- des cas nominaux,
- des cas d’erreur,
- des mocks pour simuler la base de données,
- des tests d’intégration,
- des tests E2E via Postman/Newman.

11. Planning sur 7 jours

Jour 1

- cadrage
- organisation
- contrat d’API
- squelette du projet

Jour 2

- gestion des dossiers et fichiers
- premiers écrans clients

Jour 3

- partages sécurisés
- journalisation

Jour 4

- versionnage des fichiers
- intégration côté clients

Jour 5

- sécurité
- pagination

- amélioration des erreurs

Jour 6

- stabilisation
- documentation
- tests

Jour 7

- démonstration finale
- soutenance
- bilan

12. Critères d’acceptation du MVP

Back-end

- upload chiffré fonctionnel,
- lien expiré ou révoqué géré correctement,
- versionnage opérationnel,
- quotas correctement appliqués.

Client JavaFX

- navigation fonctionnelle,
- upload avec progression,
- partage opérationnel,
- gestion des erreurs claire.

Client Web

- affichage d’un lien valide,
- affichage précis des erreurs,
- téléchargement fonctionnel,
- page responsive et accessible.

Qualité générale

- CI verte,
- documentation exploitable,
- environnement relançable,
- dépôt propre et structuré.

13. Livrables attendus

- API fonctionnelle,
- client JavaFX,
- interface Web publique,
- documentation technique,

- guide utilisateur,
- documentation d’exploitation,
- tests,
- démonstration finale,
- dépôt GitHub structuré.

14. Perspectives d’évolution

- double authentification (2FA / TOTP),
- notifications email,
- statistiques détaillées,
- rôles avancés,
- monitoring,
- export / suppression RGPD complète,
- packaging natif JavaFX,
- internationalisation FR / EN.

15. Conclusion

Ce cahier des charges définit le cadre fonctionnel, technique et organisationnel du projet **CryptoVault**.
Il précise les objectifs attendus, les contraintes de sécurité, les fonctionnalités à développer ainsi que les modalités de réalisation du projet dans un contexte pédagogique professionnalisant.
Il sert de référence commune pour guider la conception, le développement, les tests et la démonstration finale d’une solution de coffre-fort numérique sécurisé, cohérente, maintenable et évolutive.